Coding Theory

Nate Bryerton, Arun Jannupreddy, Malik Kurtz, Dailin Li,
Rohan Radadiya, Ridge Redding, Eva Simpson

Geometry Lab Fall 2024

1 Introduction

We aim to study the methods in which we can encode messages with added redundancy so that when errors occur within the message, we can trace back what the original message was. The way we do this is by creating a mapping from the set of messages to a certain set of **codewords**. We call that set the **code**.

We will specifically look at how we can encode messages which are elements of the vector space $\mathbb{F}_q^n$ which corresponds to messages of length n with q letters in the 'alphabet' (although we must have q a power of p). We will also restrict ourselves to codes which are a **vector subspace** of $\mathbb{F}_q^n$ so that we can describe the mapping from messages to codes as a **linear transformation** between two vector spaces. This makes encoding messages and describing the code much more concise. We will continue by describing codes as either the column space of a matrix (generator matrix) or the null space of a matrix (parity-check matrix). For more info, see section 4.1.

While coding theory may sound similar to cryptography, they encapsulate two different ideas of data manipulation. The purpose of coding theory is on the reliable transmission of data over noisy communication channels, while the purpose of cryptography is to secure data against unauthorized access or tampering. Essentially, coding theory focuses on the **reliability** of data while cryptography focuses on the **security** of data.

2 Finite Fields

Definition 2.1. A *field* is a set F together with two operations denoted $+$ and $\times$, which satisfy several axioms: For arbitrary $a, b, c \in F$,

1. Associativity: $(a + b) + c = a + (b + c)$ and $(a \times b) \times c = a \times (b \times c)$.
2. Commutativity: $a + b = b + a$ and $a \times b = b \times a$.
3. Identities: There exists elements called $0, 1 \in F$ such that $a + 0 = a$ and $a \times 1 = a$ for all a.

4. Inverses: $\forall a \in F, \exists(-a) \in F$ such that $a+(-a)=0$. $\forall a \neq 0 \in F, \exists(a^{-1}) \in F$ such that $a \times a^{-1}=1$.

5. Distributivity: $a \times (b+c) = a \times b + a \times c$.

An easy example of this is the set of rational numbers with the traditional operations of addition and multiplication. It turns out that these fields do not need to be infinitely large. In the context of coding theory, we will only work with finite fields.

As an example, consider the finite field

$$\mathbb{Z}/5\mathbb{Z} := (\{0,1,2,3,4\}, +, \times)$$

Where addition and multiplication are defined modulo 5. Then it's clear that $-x = 5-x$, 0 is the additive identity, and 1 is the multiplicative identity. The multiplicative inverses are:

$$1^{-1}=1, 2^{-1}=3, 3^{-1}=2, 4^{-1}=4.$$

In general, a similar "modular finite field" $\mathbb{Z}/p\mathbb{Z}$ exists for prime sizes p.

Theorem 1. *The set $\mathbb{Z}/p\mathbb{Z}$ for prime p under traditional definitions of addition and multiplication is a field.*

Proof. Since this is a subset of $\mathbb{Z}$, Associativity, Commutativity, Distributivity still hold. Also, since this set always contains one and zero, we also have the identity axiom satisfied. Also, the additive inverse of an element x of this set is just $p-x$. All that remains is the existence of a multiplicative inverse.

We use that fact that if two integers a and b have gcd of 1, then there exist m and n such that $am+bn=1$. Choose a nonzero element of $a \in \mathbb{Z}/p\mathbb{Z}$, then use the above fact with $b=p$. Since p is prime, $\gcd(a,p)$=1, so there exist integers m,n such that $am+pn=1$, meaning $am \equiv 1 \mod p$. It may be that $m \notin \mathbb{Z}/n\mathbb{Z}$, but we can obtain a different integer with the same property in $\mathbb{Z}/p\mathbb{Z}$ by subtracting a sufficient multiple of p. □

Theorem 2. *The size of any field of the form $\mathbb{Z}/n\mathbb{Z}$ must be a prime number.*

Proof. (by contradiction) Suppose that the size is some composite integer $n = a \times b$ with $a \neq b$ and $a, b \geq 2$. Then in the field $\mathbb{Z}/n\mathbb{Z}$, $a \times b = 0$. But then $b = (a^{-1} \times a) \times b = a^{-1} \times (a \times b) = a^{-1} \times 0 = 0$, which is a contradiction. □

The following Theorem is an important fact:

Theorem 3.

1. *Every finite field has p^k elements for some prime p and positive integer k.*

2. *For every prime p and positive integer k, there exists a finite field with p^k elements.*

3. *Any two finite fields of the same size are isomorphic, meaning that there exists a bijective function between them which preserves the field operations.*

We will not prove the above theorem as it involves quite a bit of algebra that would need to proved first and the proof is not necessary to our study of coding theory. This theorem means that every finite field of size p^k can be constructed from the field of size p. One way to do this is consider the set of polynomials of degree $k-1$ whose coefficients are in the field of size p. Operations are defined the normal way, except that we take the remainder of the polynomial modulo some irreducible (no zeroes) polynomial of degree k. The fact that this in fact forms a field is non-trivial, but can be proven with arguments similar to those in the proof of Theorem 1.

Example 1: A finite field with 8 elements

As an example of the previous statement, we will describe the field of size 8 using polynomials. First we need an irreducible polynomial of degree 3 with coefficients in the field $\mathbb{F}_2$. We will use $q(x) = x^3 + x + 1$, notice that this is irreducible since $q(1) = 1$ and $q(0) = 1$ and this q has no roots. Now the elements of our field are $\{0, 1, x, x+1, x^2, x^2+1, x^2+x, x^2+x+1\}$ and we show a partial table of the multiplication operation (using $x^3 = x + 1$ modulo $q(x)$):

$\times$	0	1	x	$x+1$	x^2	x^2+x	x^2+1	x^2+x+1
0	0	0	0	0	0	0	0	0
1	0	1	x	$x+1$	x^2	x^2+x	x^2+1	x^2+x+1
x	0	x	x^2	x^2+x	$x+1$	x^2+x+1	1	x^2+1
$x+1$	0	$x+1$	x^2+x	x^2+1	x^2+x+1	1	x	x^2
x^2	0	x^2	x^2+x	1	x^2+x+1	x	$x+1$	x^2+1
x^2+x	0	x^2+x	1	x^2	$x+1$	x^2+1	x	x^2+x+1
x^2+1	0	x^2+1	x^2+x+1	1	x	$x+1$	x^2	x^2+x
x^2+x+1	0	x^2+x+1	1	x^2+x	x^2	x	x^2+1	$x+1$

Example 2: Standard Equations over Finite Fields

Additionally, when analyzing various algebraic equations over finite fields, it is important to note that solutions to these equations often show different patterns when compared to their traditional solutions over all real numbers.

Lemma 1. *The equation $x^2 + 1 = 0$ has solutions in prime fields $\mathbb{F}_p$, where $p = 4n + 1$.*

Proof. Consider the given equation:

$$x^2 + 1 = 0. \tag{1}$$

Over the real numbers, this equation yields no solution. However, if we instead look for solutions to the equation over finite fields $\mathbb{F}_p$, where our chosen field is modulo p, we obtain very different results. Adjusting the equation to include modular arithmetic, we obtain the following:

$$x^2 + 1 \equiv 0 \mod p, \tag{2}$$

From this new equality, we can deduce the following:

$$x^2 \equiv -1 \mod p. \tag{3}$$

Since every element of $\mathbb{F}_p$ and -1 is co-prime to p, we can apply Fermat's little theorem (see Ref. 4) to obtain the following:

$$(-1)^{p-1} \equiv -1 \mod p, \tag{4}$$

Factoring the above equation, we obtain:

$$((-1)^{\frac{p-1}{2}} - 1)((-1)^{\frac{p-1}{2}} + 1) \equiv 0 \mod p. \tag{5}$$

From this, we can devise the following two cases in which p yields a solution to the equation:

$$(-1)^{\frac{p-1}{2}} \equiv 1 \mod p, \tag{6}$$

$$(-1)^{\frac{p-1}{2}} \equiv -1 \mod p. \tag{7}$$

However, by applying Fermat's little theorem once again, we can eliminate possible solutions derived from the second case (equation 7) by showing the following (note that x and p are co-prime because x is an element in $\mathbb{F}_p$):

$$(-1)^{\frac{p-1}{2}} \equiv (x^2)^{\frac{p-1}{2}} \mod p \equiv x^{p-1} \mod p, \tag{8}$$

$$x^{p-1} \equiv 1 \mod p. \tag{9}$$

Thus,

$$(-1)^{\frac{p-1}{2}} \equiv 1 \mod p. \tag{10}$$

The congruency in equation 10 is only satisfied when $\frac{p-1}{2}$ is an even integer, and thus we obtain the following solutions:

$$\frac{p-1}{2} = 2n, \tag{11}$$

Simplifying further, we obtain the following solution:

$$p = 4n + 1, \tag{12}$$

$$\therefore p \equiv 1 \mod 4. \tag{13}$$

Thus, we can see that when operating over finite fields $\mathbb{F}_p$, the standard equation $x^2 + 1 = 0$ now yields solutions over any finite field $\mathbb{F}_p$ in which p is any prime variation of $4n + 1$, where n is a positive integer. □

3 Definitions and Parameters

Recall from the Introduction that a code is a subset of A^n for some integer n and alphabet A. From now on, we will only talk about codes which are linear, and may refer to linear codes simply as 'codes'. If we wish to generalize to nonlinear codes, we will specifically say so. In all definitions, q is some power of a prime p. Every linear code C has certain fixed parameters, denoted with d, n, and k, which describe the code's ability to detect errors, the amount of space the code takes up, and the amount of information the code can process, respectively.

Definition 3.1. A *linear code* is vector subspace of $\mathbb{F}_q^n$.

Definition 3.2. The *Hamming distance* between two vectors $x, y \in \mathbb{F}_q^n$ is the number of coordinates where they differ. In other words, the Hamming distance between two vectors $x = (x_1, x_2, \ldots, x_k)$ and $y = (y_1, y_2, \ldots, y_k)$ is

$$d(x, y) = |\{i | x_i \neq y_i\}|.$$

Lemma 2. *The Hamming distance, defined above as a function $d : \mathbb{F}_q^n \times \mathbb{F}_q^n \to \mathbb{R}$, is a metric. That is, it satisfies the following three axioms:*
(i) (Positivity) $d(x, y) = 0 \iff x = y$
(ii) (Symmetry) $d(x, y) = d(y, x)$
(iii) (Triangle inequality) $d(x, z) \leq d(x, y) + d(y, z)$

Proof. (i)

$$\begin{aligned} d(x, y) = |\{i | x_i \neq y_i\}| = 0 &\iff x_i = y_i \quad \forall 1 \leq i \leq n \\ &\iff x = y. \end{aligned}$$

(ii) It's clear from the definition that d is symmetric.

(iii) Consider the sets $A := \{i | x_i \neq y_i\}, B := \{i | y_i \neq z_i\}$, and $C := \{i | x_i \neq z_i\}$ so that $d(x, y) = |A|$, $d(y, z) = |B|$, and $d(x, z) = |C|$. By transitivity,

$$x_i = y_i \text{ and } y_i = z_i \implies x_i = z_i.$$

So the contrapositive

$$x_i \neq z_i \implies x_i \neq y_i \text{ or } y_i \neq z_i$$

is also true. This means that $C \subseteq A \cup B$ and thus $|C| \leq |A| + |B|$. □

Definition 3.3. The *minimum distance* d of a code C is just that, meaning

$$d = \min_{x \neq y \in C} d(x, y).$$

This represents how resilient the code is to errors.

Definition 3.4. The *weight* $wt(c)$ of a codeword is its Hamming distance from 0. That is,

$$wt(c) := d(c, 0)$$

The minimum distance of a linear code can be computed in another way, which in some cases is easier.

Lemma 3. *The minimum distance d of a code C is the same as the minimum weight of C. That is,*

$$d = \min_{x \neq 0 \in C} d(x, 0).$$

.

Proof. By the definition of minimum distance, it's clear that $\min_{x \in C} d(x, 0) \geq d$. Now suppose, by way of contradiction, that $\min_{x \in C} d(x, 0) > d$, then there would exist $y_1, y_2 \in C$ such that $min_{x \in C} d(x, 0) > d(y_1, y_2)$. Notice that $d(y_1 - y_2, 0) = d(y_1, y_2)$ since a a coordinate of $y_1 - y_2$ is zero if and only if the coordinate of y_1 matches that of y_2. Now by linearity, $y_1 - y_2$ is also in C. But then $d(y_1 - y_2, 0)$ is less than the minimum weight, is a contradiction. □

Definition 3.5. The *length* n of a code C is the dimension n of the ambient vector space. This represents how much space each codeword takes up.

Definition 3.6. The *dimension* k of a code is the dimension of C as a vector space itself. This represents the amount of information that the code can encode.

Definition 3.7. The *rate* R of a code is simply k/n and represents how much information the code provides relative to how much space it takes up.

Definition 3.8. The *relative distance* δ of a code is simply d/n and represents the proportion of bits that are contributing towards the minimum distance of the code.

To concisely describe the relevant properties of a code C, we may write "C is an $[n, k, d]_q$ code" where $n, k, d, .$ and q are defined as above.

4 Codes

Before we delve into how we can create codes, we begin by introducing a naive way of encoding information and use it as a recurring example throughout this section.

Definition 4.1. A repetition code is the image of a repetition encoding, which is a function f defined by the following:

$$f : \mathbb{F}_q \to \mathbb{F}_q^n$$
$$f(x) := (x, x, \ldots, x)$$

Essentially, it just creates n copies of the initial message.

Lemma 4. *A repetition code with length n and alphabet size q is a $[n, 1, n]_q$ code. It's rate is asymptotically 0 (meaning it approaches 0 as the length of the code increases) and its relative distance is 1.*

Proof. The dimension of the repetition code is 1 because its basis is just a single vector $(1, 1, \ldots, 1)$. The minimum distance is n because any two codewords differ at all n coordinates. Then the rate is $R = k/n = 1/n \to 0$ and the relative distance is $\sigma = d/n = 1$ □

4.1 Describing Codes

Two fundamental results from linear algebra are the following:

Theorem 4. *The image and kernel of a linear map applied to a vector space are each vector spaces*

What this means is that if we can describe a code as the range of some linear transformation applied to $\mathbb{F}_q^k$ (since the code has dimension k), then we can compute codewords particularly easily. This leads to the following:

Definition 4.2. A *generator matrix* G for a code C is a matrix whose image is the code C, in other words

$$C = \{\mathbf{Gm} | \mathbf{m} \in \mathbb{F}_q^k\}$$

which means G has k columns and n rows. Here we force the rows of G to be linearly independent and treat elements of $\mathbb{F}_q^k$ as column vectors.

Coming back to the repetition code, one generator matrix for that code would be

$$\begin{pmatrix} 1 \\ 1 \\ \ldots \\ 1 \end{pmatrix}.$$

Note that there is not necessarily a *unique* generator matrix since another generator matrix in this instance would be

$$\begin{pmatrix} 2 \\ 2 \\ \ldots \\ 2 \end{pmatrix}.$$

The second part of Theorem 4 lends us to the following alternative description of a code:

Definition 4.3. A *parity check matrix* H for a code C is a matrix whose null space is the code C, in other words

$$C = \{\mathbf{x} \in \mathbb{F}_q^n | \mathbf{Hx} = 0\},$$

which means H has n columns and $n - k$ rows. Here we force the rows of H to be linearly independent.

A parity check matrix for the repetition code would look like

$$\begin{pmatrix} 1 & -1 & 0 & 0 & \dots & 0 \\ 0 & 1 & -1 & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & 0 & 1 & -1 \end{pmatrix}$$

because for this times a vector to be 0, every adjacent paid of elements would have to be the same. This is just another way of describing the repetition code. Having these compact descriptions of codes is extremely useful, because it provides an easy way to find codewords (multiply something by generator matrix), or correct errors (see what the received word times the parity check matrix is/ see if it is 0).

Theorem 5. *Every linear code C has a generator matrix G*

Proof. Since C is a linear code, it is a vector subspace of $\mathbb{F}_q^n$. From linear algebra, we have that there must exists a basis β for C of size $k \leq n$. This means that the code C can be described as the set of linear combinations of the vectors in β. This set is precisely the range of the matrix whose columns are the basis vectors, this is the generator matrix for C and it has k columns and n rows. □

Before we can prove that every linear code has a parity check matrix, we first define the dual of a code:

Definition 4.4. The dual $C^\perp$ of a code C is defined as

$$C^\perp := \{\mathbf{x} \in \mathbb{F}_q^n | \ \langle x, c \rangle := x_1c_1 + \cdots + x_nc_n = 0 \ \forall c \in C\}.$$

Theorem 6. *The dual $C^\perp$ of a linear code C is a linear code.*

Proof. If $x, y \in C^\perp$, then $< x + y, c >= (x_1 + y_1)c_1 + \cdots + (x_n + y_n)c_n =< x, c > + < y, c >= 0$, so $x + y$ is also in $C^\perp$.

If $x \in C^\perp$, then for $k \in \mathbb{F}_q$, $< kx, c >= kx_1c_1 + \cdots + kx_nc_n = k < x, n >= 0$, so kx is also in $C^\perp$. Therefore $C^\perp$ is a vector space and is thus a linear code. □

This means that the dual of every code has a generator matrix, by Theorem 5. As we will see, this matrix is related to our original code.

Theorem 7. *The transpose of a generator matrix H for the dual of a linear code $C^\perp$ is a parity check matrix for the linear code C. Similarly, the transpose of a parity check matrix G for the dual of a linear code $C^\perp$ is a generator matrix for the linear code C.*

Proof. Since H is the generator matrix for the dual of the code, we have

$$< Hx, c >= 0 \quad \forall c \in C, x \in \mathbb{F}_q^n.$$

Setting $x_i = e_i$ to be the $\mathbf{0}$ vector with a 1 at position i for each $1 \leq i \leq n$, this means that the dot product of every column of H with any codeword of C is 0.

This means the dot product of every row of H^T with any codeword of C is 0, which is the definition of a parity check matrix. By applying the first part of the theorem to the generator matrix of the code itself, the second part of the theorem follows. □

To see how this works, examine the dual of the repetition code. By Theorem 7, the dual code has a parity check matrix given by

$$\begin{pmatrix} 1 & 1 & \dots & 1 \end{pmatrix}.$$

This means that the dual code is the set of vectors in $\mathbb{F}_q^n$ where the sum of coordinates is 0. This is only one restriction, and it can be satisfied by choosing the appropriate last coordinate for a given first $n-1$ coordinates. Thus the dual of the repetition code is a $[n, n-1, 2]_q$

Together, these theorems mean that every linear code can be described by a generator matrix, and by a parity check matrix.

4.2 Error detection/correction ability

The ability of a code to detect errors or correct errors is directly a function of the minimum distance d of the code. These functions are given below:

Lemma 5. *Let C be a code with minimum distance d and length n, then*

(a) the number of errors that the code can detect is

$$\epsilon_d = d - 1.$$

(b) the number of errors that the code can correct is

$$\epsilon_c = \left\lfloor \frac{d-1}{2} \right\rfloor.$$

Proof. (a) For C to be able to detect ϵ_d errors, every two codewords $x, y \in C$ must be such that $d(x, y) > \epsilon_d$. Otherwise, we would have $d(x, y) \leq \epsilon_d$ and x could be reached from y via ϵ_d errors, so the code would not detect such an error. Therefore, it is necessary and sufficient to have a minimum distance of $\epsilon_d + 1$.

(b) For C to be able to correct ϵ_c errors, every two codewords $x, y \in C$ must be such that $n \geq d(x, y) > 2\epsilon_c$. Otherwise, we would have $d(x, y) \leq 2\epsilon_c$. Then by changing the first ϵ_c coordinates of x that don't match y to the corresponding y coordinate, we create an element $z \in \mathbb{F}_q^n$ with $d(x, z) = \epsilon_c$ and $d(y, z) = d(x, y) - \epsilon_c \leq \epsilon_c$. But since z is within ϵ_c of both x and y, the code will not be able to determine whether the initial codeword was x or y. Therefore, it is necessary and sufficient to have a minimum distance of $2\epsilon_c + 1$.

□

4.3 Types of Codes

Having defined the general characteristics of linear codes, we will now introduce a few specific codes and their unique properties.

4.3.1 Example 1: Hamming code

The Hamming code is an error-correcting code that can detect and correct single-bit errors. We will focus on the binary version of it here, which encodes $2^r - r - 1$ data bits into a $(2^r - 1)$-bit codeword by introducing r parity bits.

There are many different versions of the Hamming code, but they are all essentially the same, with the coordinates shuffled around. The easiest way to define the Hamming code is as the null space of the parity check matrix which has some number r of rows, has $2^r - 1$ columns, and whose ith column is i in binary. For example, if we let $r = 3$, the parity check matrix is

$$\begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

Importantly, none of the columns match and none of the columns are 0. The length of the code is then $n = 2^r - 1$, and the dimension is $k = n - r = 2^r - r - 1$. As the following lemma shows, the minimum distance of the Hamming code is at least 3. The minimum distance must therefore be 3 since in every Hamming code we have the codeword $(1, 1, 1, 0, \ldots, 0)$.

Lemma 6. *For a linear code C with minimum distance d and a parity check matrix H: If the columns of H are pairwise non-collinear, then $d > 2$.*

Proof. By contradiction, suppose that $d \leq 2$. That means there exist two codewords $x \neq y \in C$ which differ in two or less coordinates. That is, $y = x + e_i + e_j$ with e_i being the unit vector in the ith direction. But then since H is a parity check matrix:

$$\begin{aligned} Hy &= Hx + He_i + He_j \\ 0 &= 0 + He_i + He_j \\ He_i &= -He_j \end{aligned}$$

Since e_i and e_j are unit vectors, this means that the ith and jth column of H are collinear. This is a contradiction, so it must be that $d > 2$. □

The Hamming Code can is therefore single error-correcting, and the way we have defined the code makes this process extremely simple. If a codeword x experiences an error in position i, then the vector received will be $y = x + e_i$. Now if this is multiplied by the parity check matrix H, we get $H(x + e_i) = Hx + He_i = He_i$ which is the ith column of H. But the ith column of H is just i in binary. This means that for any single error, the position of the error in binary is Hy.

4.3.2 Example 2: Extended Golay Codes

The Extended Binary Golay Code C_{24} is a linear, error-correcting code. This particular code consists of a set of 12 information bits that are encoded into

24-bit codewords. Thus, this particular code has codewords of length 24 with a dimension of 12, as there are 12 individual information bits that can be encoded within each linearly independent codeword.

With these baseline parameters established, we can define the generator matrix of the Extended Binary Golay Code, G, as a $[12 \times 24]$ matrix consisting of a $[12 \times 12]$ identity matrix I_{12} appended to the left of another $[12 \times 12]$ matrix B_{12}. This B_{12} matrix consists of a row and column of parity bits appended to an $[11 \times 11]$ sub-matrix. The first row of the sub-matrix consists of 11 bits that correspond to the quadratic residues modulo 11, with the quadratic residues modulo eleven (i.e. bits 1, 3, 4, 5 and 9) (see Ref. 6) taking on a value of 0, while the non-quadratic residues take on a value of 1. This can also be thought of as the adjacency matrix of a 12-vertex icosahedron (see Ref. 1). Each subsequent row of the B_{12} sub-matrix is a rotation of the quadratic residues in the first row. Appending the B_{12} matrix to the identity matrix, we get the full generator matrix G to be:

$$G = [I_{12}|B_{12}] = [I_{12}] \begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \end{pmatrix}$$

The 12 rows of G form the linear subspace through which all code words are generated in the extended Golay Code, and any 12-bit message vector can be multiplied by G to form a codeword that consists of a linear combination of the rows of G.

Additionally, because each of the inner-rows of the B_{12} sub-matrix is a rotation of the previous row, the transpose of B, B^T is equal to the inverse of B, B^{-1}. Thus, the following relationship between the generator matrix G and its transpose G^T can be observed:

$$G \times G^T = 0$$

Because multiplying G by its transpose results in a zero matrix, G (and by extension the entire extended Golay Code) can be considered self-orthogonal, as all of the individual basis vectors contained within the generator matrix are orthogonal and linearly independent. This means that the extended Golay Code C_{24} must be contained within its dual code $C_{24}^{\perp}$.

Lemma 7. *The extended Golay Code C_{24} is self dual*

Proof. Knowing that C_{24} is self-orthogonal, we can additionally prove that the extended Golay Code is self-dual. To do this, recall that the relationship between a parity-check matrix H of a code C with length and dimension $\{n, k\}$ with generator matrix G can be defined as follows. (see Ref. 8)

First, the standard form of G can be represented with the expression below:

$$G = [I_k|B]$$

Next, recall that the following relationship between the generator and parity check matrices of any code C must be true:

$$GH^T = B - B = 0$$

Thus, we can define the parity check matrix H as:

$$H = [-B^T|I_{(n-k)}]$$

The above equation reveals a some important features of the extended Golay code. First, it is important to note that because the extra parity bits that were added to the sub-matrix of B_{12}, the length of the Golay Code C is extended to 24-bits, meaning that the identity matrix within H is a $[12 \times 12]$ identity matrix. From this, we can obtain the following:

$$H = [-B^T|I_{(12)}] = [I_{(12)}|B] = G$$

Thus, the parity check matrix H of the extended Golay Code C is equivalent to its generator matrix G. This means that the dual code $C^\perp$ of C is equal to C. This is a key distinction from C being self-orthogonal, as we have now proved that C is equivalent to $C^\perp$ instead of simply being contained within the null space of $C^\perp$ (see Definition 6 for the standard definition of a dual code). Thus, C can be defined as self-dual, meaning that its generator matrix G can be utilized to both encode and decode any 12-bit message vector. □

At this point, it is also important to note that the minimum weight of each codeword within C is 8 (see Ref. 2), and from Lemma (5), we can deduce that the extended Golay Code can correct up to 3 errors.

4.3.3 Single-Half Error Correction For the Extended Golay Code

The extended Golay Code has multiple error correcting variations, the first of which occurs when up to 3 errors exist within one half of the received 24-bit codeword r_e.

For example, take the following codeword vector r:

$$\begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \end{pmatrix}$$

This codeword was generated by multiplying a 12-bit vector by the generator matrix G. Now, let's add an error vector e by flipping the 11th bit of r to obtain an incorrect code word r_e, where $r_e = r + e$:

$$\begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \end{pmatrix}$$

Now, with the error embedded within r_e, we can multiply the parity check matrix H by the transpose of the received codeword r_e^T to determine where the errors are (note that the output of this particular matrix multiplication is commonly referred to as the syndrome vector):

$$Hr_e^T = [I_{12}|B]r_e^T = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \mathbf{1} \\ 0 \end{pmatrix}$$

Note that we initially multiplied the original 12-bit message by the generator matrix and then the multiplied the received codeword by an equivalent parity-check matrix. Because each codeword is a linear combination of each independent row of G, the decoded syndrome vector would be equal to 0 if the received codeword r_e was error-free. However, the resultant syndrome vector in this particular example has a flipped bit in the 11th position, which correctly corresponds to the error vector e that was added to r before decoding. Thus, we can simply flip the 11th bit of the 24-bit codeword to fix the existing error. This method is applicable for up to 3 flipped bits, so long as all three of those bits are concentrated in either the first half or the second half of the codeword.

4.3.4 Dispersed Error Correction For the Extended Golay Code

When errors are dispersed throughout the codeword, a different correction method needs to be utilized.

To begin, recall the encoded 24-bit message r from section 4.3.3. We will once again add an error vector e to this message, but this time the error vector will flip the first and last bits of r, resulting in the following vector for r_e:

$$\begin{pmatrix} \mathbf{0} & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & \mathbf{0} \end{pmatrix}$$

Now, when we compute the syndrome vector, we obtain the following:

$$Hr_e^T = [I_{12}|B]r_e^T = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

This syndrome vector is largely different from the vector in Section 4.3.3, and thus a different error correction method must be used. Recall from Section 4.3.1 that when a given codeword experiences an error in position i, the resulting syndrome vector will correspond to the ith column of the parity check matrix. This can be applied directly to the above syndrome vector. However, it is important to note that r_e has a second error in addition to the error in position i (in this case i corresponds to the 24th bit), so r_e will not perfectly correspond to a particular column of the parity check matrix H. Instead, r_e will be equal to the ith column of H in all positions except one, which will reveal the location of the second error.

Comparing the syndrome vector to H, we can see that r_e is equal to the 24th column of H in all positions except the first position. This means that within the first half of the code, there is an error in the first bit. For the second error, since our codeword corresponds to the 24th column of H, our codeword must have experienced an error in position 24, and the 24th bit must be flipped to correct the codeword. These two detected errors correctly correspond to the errors that we added to r before calculating the syndrome vector, showing the error-correcting capabilities of the extended Golay Code when the errors are dispersed.

4.3.5 Binary Goppa Codes

Typically, a Binary Goppa Code is defined by a list

$$L = a_1 + a_2 + \cdots + a_n$$

of distinct integers n over finite field $\mathbb{F}_q$, where $q = 2^m$, which is also called the support. It must also have a square-free polynomial $g(x) \in \mathbb{F}_q[x]$, with degree t such that $g(a) \neq 0$ for all $a \in L$. Finally, $g(x)$ should also be irreducible over $\mathbb{F}_q$, meaning it cannot be reduced to its roots over $\mathbb{F}_q$. So, the corresponding Binary Goppa Code can be defined as

$$\gamma(L, g) = \{\mathbf{x} \in \mathbb{F}_q^n | \sum_{i=1}^{n} \frac{c_i}{x - L_i} \equiv 0 \mod g(x)\}$$

The code defined by (g, L) posseses a dimension of at least n - mt, and a distance of at least 2t+1. This means that it will be able to encode messages with a length of at least n - mt that are comprised of codewords of at least size n, and will be able to correct at least

$$\frac{(2t+1)-1}{2}$$

errors.

4.3.6 Reed-Solomon Codes

Reed-Solomon Codes are a form of error-correcting code that fits a sequence of n elements to a unique polynomial of degree $n-1$. For error-correcting ability, an additional k redundant points are evaluated from the polynomial and appended to the sequence. As a result, if k or less points suffer an identifiable error, the decoder can rebuild the unique polynomial from any valid n points, and evaluate the polynomial at corresponding points to recover lost information.

The primary method of fitting points to a polynomial $f(x)$ is through Lagrange Interpolation, as follows:

$$l_i(x) = \prod_{j=1, j\neq i}^{k} \frac{x - x_j}{x_i - x_j}$$

$$f(x) = \sum_{i=1}^{k} y_i l_i(x)$$

For each evaluation point, $x = a$, that corresponds to a fitted element, $f(x = a)$, we assign a sub-polynomial, $l_{i=a}$ such that $l_i(x) = 1$ at $x = a$ and 0 at every other evaluation point. These "Basis Polynomials" can be reused for consecutive encodings and decodings with equivalent evaluation points. We can then scale each Basis Polynomial by the fitted element in the sequence and sum all scaled polynomials together to form the polynomial fit. Any additional points can then be evaluated from this polynomial.

On the decoding end, the same method can be applied over any n points, as long as n valid points exist. Since the fit is unique, the decoder will produce the identical equation as the encoder. Any invalid points can then be recomputed from the resulting polynomial.

5 Bounds

In this section, we place some limits on how powerful our code can be. We have already seen that a linear code C of length n, dimension k and minimum distance d is efficient if k is large (with respect to n) and corrects many errors if d is large (with respect to n). However, given n and k, how large can d be? Or, given n and d, how large can k be? This is where bounds come into play.

5.1 Singleton Bound

The first of our bounds is based on the codes being subspaces of $\mathbb{F}_q^n$.

Theorem 8. *Let C be a linear code of length n, dimension k, and minimum distance d over $\mathbb{F}_q$. Then the Singleton Bound is*

$$d + k \leq n + 1.$$

Proof. We define the subset $W \subseteq \mathbb{F}_q^n$ as:

$$W := \{\mathbf{a} = (a_1, \ldots, a_n) \in \mathbb{F}_q^n \mid a_d = a_{d+1} = \cdots = a_n = 0\}.$$

For any $\mathbf{a} \in W$, the weight $\mathrm{wt}(\mathbf{a}) \leq d-1$. Therefore, $W \cap C = \{0\}$. From linear algebra, this means

$$n \geq \dim(W + C) = \dim W + \dim C,$$

where $W + C := \{\mathbf{w} + \mathbf{c} \mid \mathbf{w} \in W, \mathbf{c} \in C\}$. Since $\dim W = d-1$ and $\dim C = k$, this implies:

$$d - 1 + k \leq n.$$

Thus:

$$d + k \leq n + 1.$$

□

A code which achieves this equality is called Maximum Distance Separable (MDS). One example of this is the repetition code. Recall that the repetition code is a $[n, 1, n]_q$ code, meaning $k = 1$ and $d = n - 1$. Plugging this into the Singleton bound, we get

$$\begin{aligned}(n-1) + 1 &\leq n + 1 \\ n + 1 &\leq n + 1,\end{aligned}$$

which is actually just an equality. This means that neither the dimension nor the minimum distance of the code can be improved without increasing its length. Importantly, changing the size of the alphabet q does nothing to change this. A more useful MDS code is the Reed-Solomon code (see Reed-Solomon Codes).

5.2 Hamming Bound

We have another bound that makes a statement about the number of codewords in a code, and it can be used to demonstrate that Hamming codes are in a sense *perfect*.

Theorem 9. *Let $A_q(n, d)$ be the number of codewords for a code C with alphabet of size q, length n, and minimum distance d. Then*

$$A_q(n, d) \leq \frac{q^n}{\sum_{k=0}^{t} \binom{n}{k}(q-1)^k}, \qquad \textit{with} \qquad t = \left\lfloor \frac{d-1}{2} \right\rfloor.$$

Proof. The strategy here will be to estimate the number of elements of $\mathbb{F}_q^n$ by finding the size of the Hamming balls around each codeword.

Consider each codeword $c \in C$ and the set of all words which differ by at most $t = \lfloor \frac{d-1}{2} \rfloor$ coordinates from C. These sets are called *Hamming balls* around each codeword. Since the minimum distance between codewords is d, these Hamming balls must be disjoint. For suppose that a word was in two different Hamming balls, then it would be a distance of at most t from each of two different codewords. But then by the triangle inequality, the distance between those two codewords must be at most $2t = 2\lfloor \frac{d-1}{2} \rfloor \leq d-1 < d$, violating the minimum distance being d.

Next we count the number of words in each Hamming ball. We can change up to t of the coordinates while staying in the Hamming ball. So for each k from 0 to t, we count the number of words of distance k from the central codeword. We much choose k coordinates to be flipped to one of $(q-1)$ other choices, therefore the number of words of distance k from the codeword is $\binom{n}{k}(q-1)^k$. Therefore the number of words in the Hamming Ball is $\sum_{k=0}^{t} \binom{n}{k}(q-1)^k$.

Because each Hamming Ball is disjoint, the sum of the number of words in each Hamming ball over all codewords will count each word at most once. This means the sum will be less than or equal to the actual number of words. Recalling that $A_q(n,d)$ is the number of codewords in the code, we have

$$A_q(n,d) \sum_{k=0}^{t} \binom{n}{k}(q-1)^k \leq q^n$$

Rearranging this brings us to the Hamming Bound. □

An interesting thing to note here is that this bound applies for any code, not just linear ones. Now we consider the Hamming Code. Recall that the best Hamming Code has a parity check matrix with r rows and n columns where $n = 2^r - 1$. The dimension of the code is $n - r = 2^r - r - 1$. The q here is 2 because we work in binary, and recall that this code has a minimum distance of 3. Then we have $t = \lfloor 3/2 \rfloor = 1$. $A_q(n,d) = 2^k = 2^{2^r-r-1}$ since the code is linear. Substituting all of the is into the Hamming Bound, we get:

$$\begin{aligned}
A_q(n,d) &\leq \frac{q^n}{\sum_{k=0}^{t} \binom{n}{k}(q-1)^k} \\
2^{2^r-r-1} &\leq \frac{2^{2^r-1}}{\binom{2^r-1}{0}(2-1)^0 + \binom{2^r-1}{1}(2-1)^1} \\
2^{-r} &\leq \frac{1}{1+2^r-1} \\
2^{-r} &\leq 2^{-r},
\end{aligned}$$

which is actually an equality. We call such codes *perfect*, because every single vector in $\mathbb{F}_q^n$ is within a Hamming Ball, meaning the code uses up the entirety of $\mathbb{F}_q^n$.

6 References

1. Baez, J. (2015). Golay Code. Visual insight. American Mathematical Society. from https://blogs.ams.org/visualinsight/2015/12/01/golay-code/.

2. The Binary Golay Code—UC San Diego Jacobs School of Engineering. (n.d.). https://cseweb.ucsd.edu/classes/fa21/ece259-a/Lectures/lecture09-handout.pdf.

3. Couvreur, A. (2020). Introduction to Coding Theory.

4. Fermat's Little Theorem. (n.d.). Art of Problem Solving. from artofproblemsolving.com

5. Field (mathematics)—Wikipedia. (n.d.). from https://en.wikipedia.org/wiki/Field(mathematics)

6. Cook, J. D. (2022). Golay codes. from https://www.johndcook.com/blog/2019/10/18/golay-code/.

7. Judson, T. W. (2020). Abstract Algebra: Theory and Applications.

8. Parity-check matrix—Wikipedia. (n.d.). from https://en.wikipedia.org/wiki/Parity-check-matrix.